

Planning of collision free and feasible robot motions

Professor Henrik Gordon Petersen

University of Southern Denmark

Email: hgp@mmmi.sdu.dk

URL: <https://www.sdu.dk/en/forskning/sdurobotics>

A bit about me...

- I have a PhD in Applied Mathematics
- Worked with molecular dynamics simulations of protein folding during my PhD (ancient history 😊)
- I have been a robotics scientist since 1991
- Focus on robotics technologies for industrial manufacturing
- Co-founder of a spinoff company in 1996 aimed at motion planning
- Participated in many national and European projects – typically involving industry-academia collaboration
- My field is within mathematical modelling, planning and control of robotic systems.
- Some pictures...

Worked with robotics since the early 1990's



Odense Steelshipyard (OSS)



AMROSE (Autonomous Multiple Robot Operations in Structured Environments). Project with OSS (1991-1997)

Automatic programming of welding operations and movements between weldings (up to 5 hours of execution). Based on CAD models and corrections by touching operations.

Manufacturing Academy of Denmark

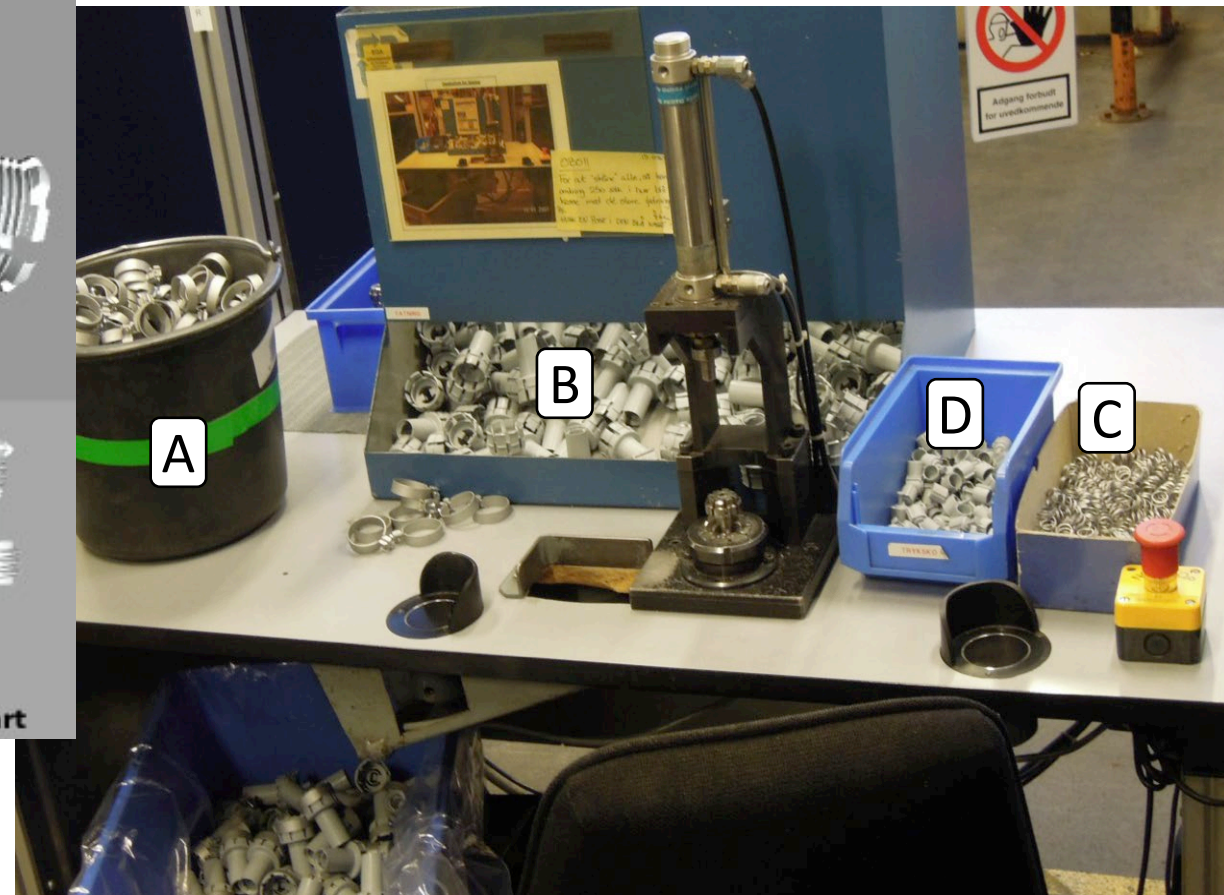
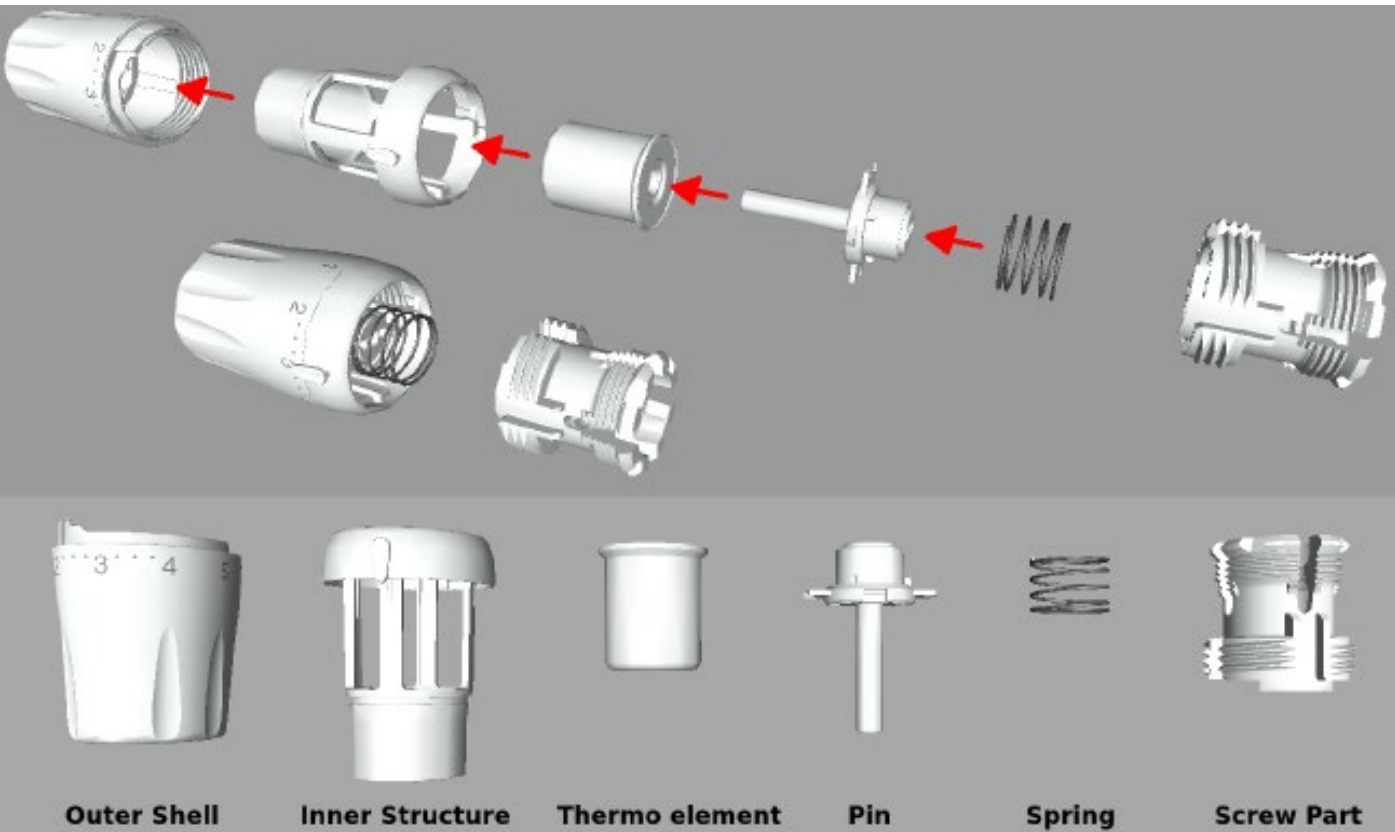
Agile production systems

Today, many manufacturing processes are manual, for example product assembly. This is due to the majority of Danish manufacturing companies producing small batches of customer specific products where traditional automation solutions are not suited. To maintain competitiveness and ensure efficiency, Danish manufacturing companies want to optimize their production and utilization of human resources.

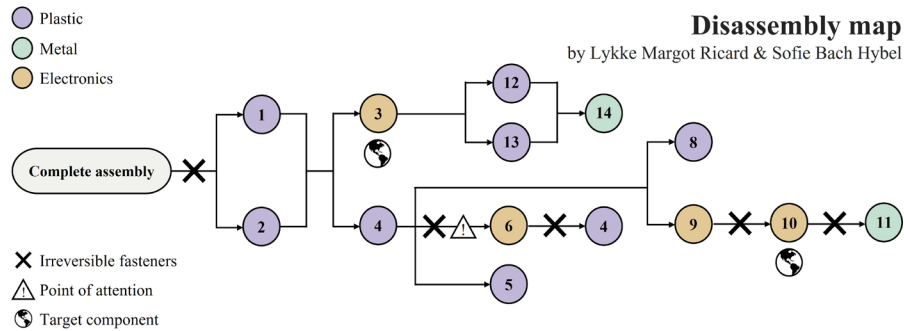


Listen to the leader of the strategic effort Agile Production Systems Henrik Gordon Petersen explain the effort.

Low volume assembly using robots



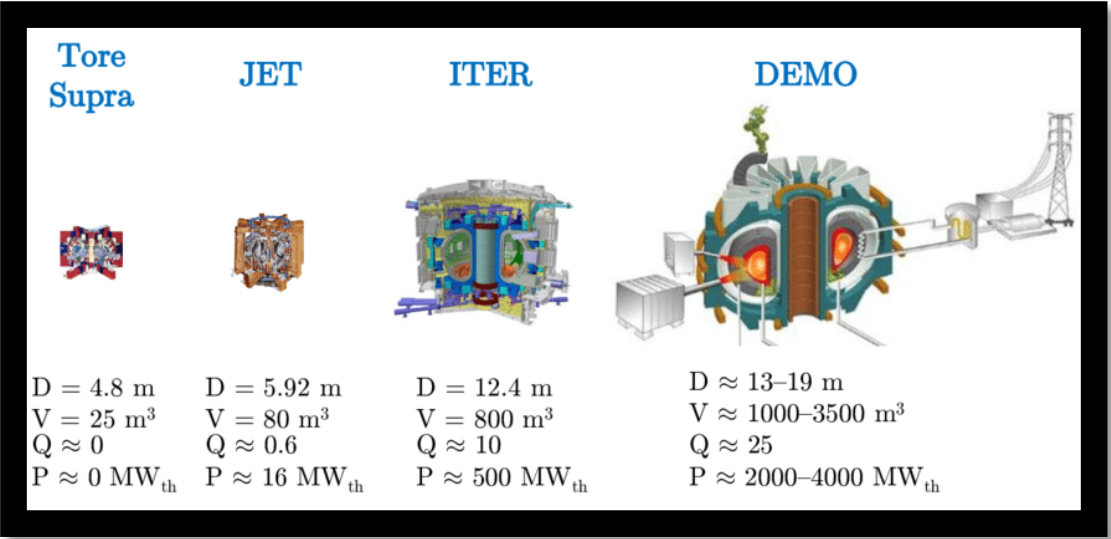
Robotics for the green transition



Automated disassembly of products for component reuse

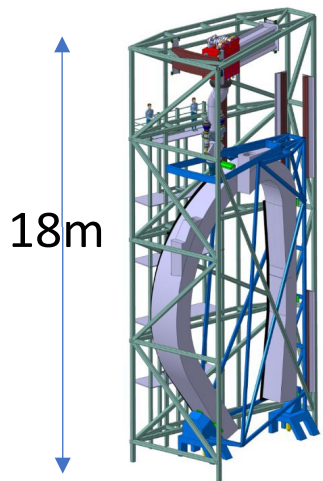


Automated layup of fiber plies on wind turbine blades



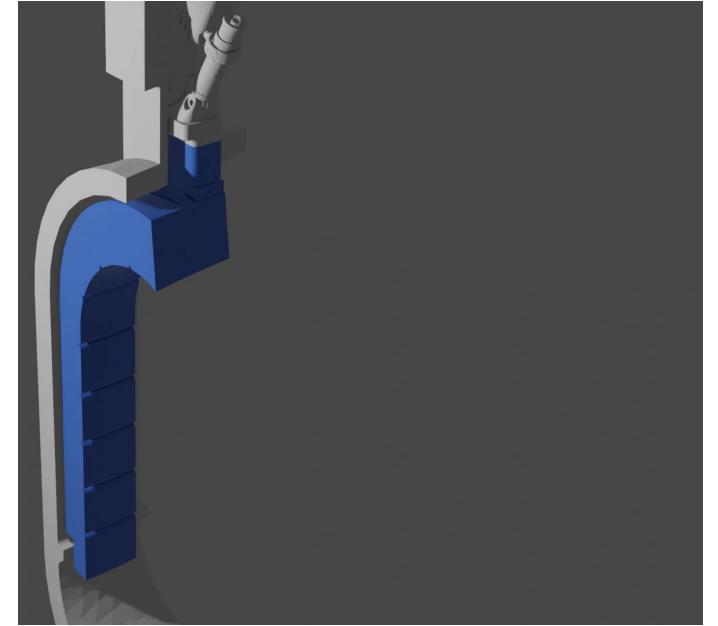
Robotic maintenance of fusion reactors

ITER is under construction



Large-scale testing and proof-of-principle, including gripping, preloading, disengagement from supports, re-engagement into supports, impact of residual magnetic attraction force, release of stiction on contact surfaces.

Facility at the Port of Odense (2026)



Design a planning and control system that ensures no collisions during the blanket removal despite ***partially unknown deflections and a very cluttered environment.***

I am a passionate Liverpool supporter



Aug. 12th : Planning of collision free and feasible robot motions

08:30 – 10.15 Lecture 1: Basic planning and trajectory concepts (HP)

- Cartesian space and configuration space
- Parametrizations of these spaces
- Mapping between spaces
- Linear interpolation with parabolic blends
- Estimating cycle time costs
- Why plan and interpolate in configuration space

10:30 – 12.00 Lecture 2: Trajectory planning for Robot Manipulators and Grasping (AK)

- Interpolation for trajectories represented in Cartesian space
- Cubic polynomial interpolation
- Robotic grasping
- Defining trajectories by Learning

12:00 – Lunch break

13:00 – 14.30 Lecture 3: Finding collision free paths (HP)

- Point to point planning
- Distance metric
- Resolution property
- Grid based and random points based sampling
- Sample based planning methods in a nutshell
- Collision testing and distance computations
- Finding shortest paths in graphs
- Example of a planning method

14:30 – Coffee break

15:00 – 16.30 Lecture 4: Behavior based robotics for easy programming (KS)

16:30 – Project work

Aug. 12th : Planning of collision free and feasible robot motions

08:30 – 10.15 Lecture 1: Basic planning and trajectory concepts (HP)

- Cartesian space and configuration space
- Parametrizations of these spaces
- Mapping between spaces
- Linear interpolation with parabolic blends
- Estimating cycle time costs
- Why plan and interpolate in configuration space

10:30 – 12.00 Lecture 2: Trajectory planning for Robot Manipulators and Grasping (AK)

- Interpolation for trajectories represented in Cartesian space
- Cubic polynomial interpolation
- Robotic grasping
- Defining trajectories by Learning

12:00 – Lunch break

13:00 – 14.30 Lecture 3: Finding collision free paths (HP)

- Point to point planning
- Distance metric
- Resolution property
- Grid based and random points based sampling
- Sample based planning methods in a nutshell
- Collision testing and distance computations
- Finding shortest paths in graphs
- Example of a planning method

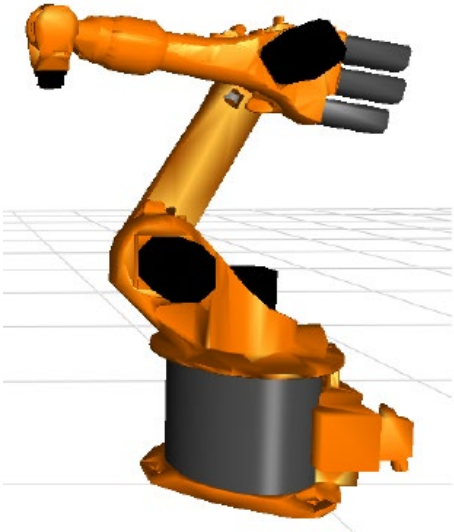
14:30 – Coffee break

15:00 – 16.30 Lecture 4: Behavior based robotics for easy programming (KS)

16:30 – Project work

Robot Manipulators

A 6DOF Kuka Robot



A 6DOF UR Robot

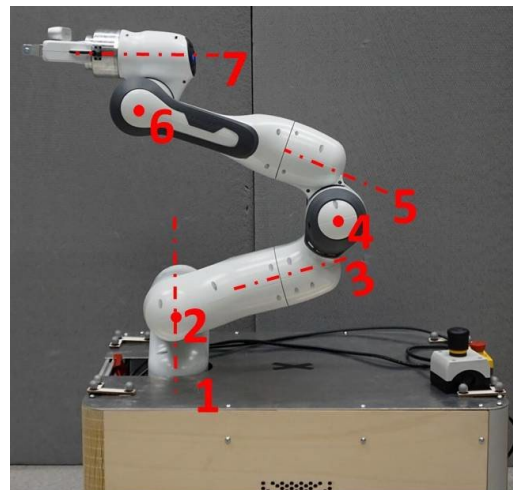


A 6DOF Fanuc Robot

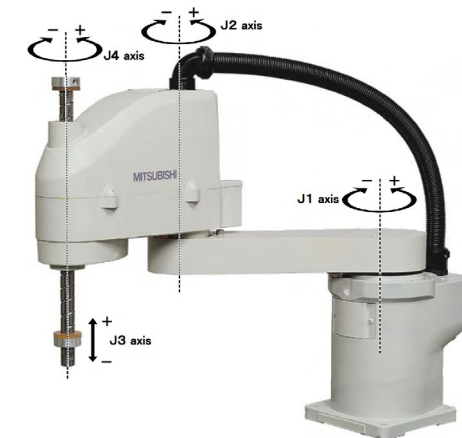


DOF=Degrees Of Freedom
=number of joints
=N

A 7DOF Franka robot



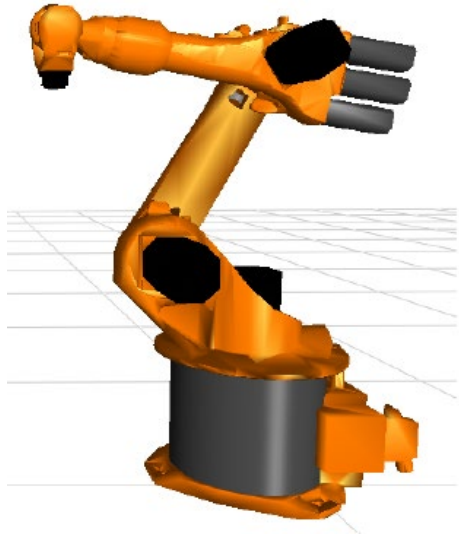
A 4DOF Scara robot



A 5DOF Fanuc Robot



Cartesian Space and Configuration Space for robot manipulator arms



A 6DOF robotic manipulator

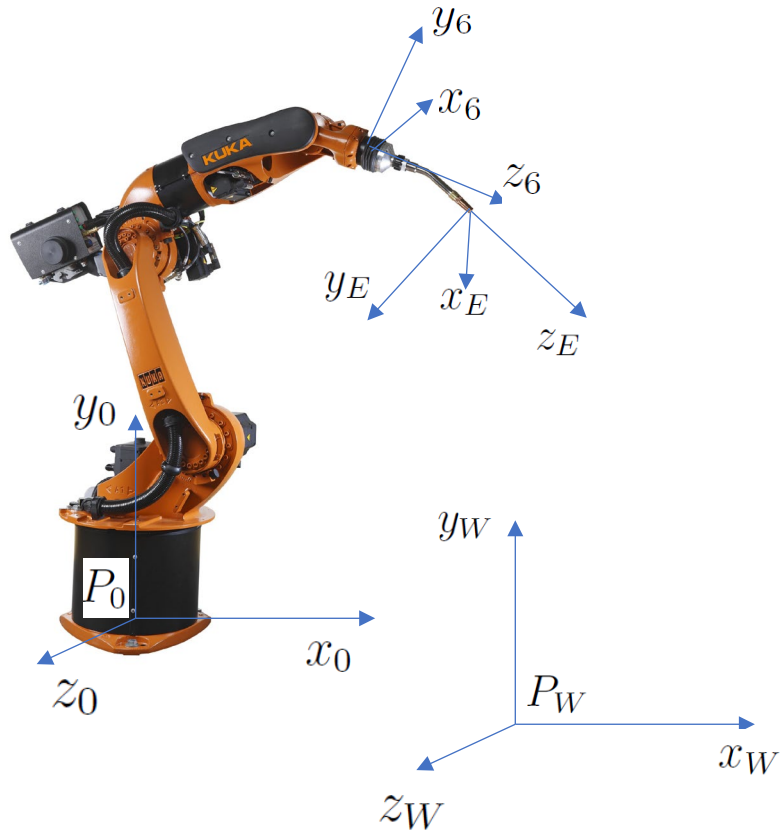
The *Cartesian space* X is defined as the set of all positions and orientations that the **outermost link can be in**.

The *configuration space* $\mathcal{C}$ is defined as the set of all configurations that the robot can be in. Each point in $\mathcal{C}$ represents a set of **positions and orientations of each of the rigid components** that are feasible with respect to the joint constraints.

We model the manipulator arm as consisting of a set of rigid links connected by joints.

Cartesian space and configuration space as mathematical sets

A 6DOF robotic manipulator (N=6)



World frame W and end effector frame E.
Robot base frame 0 and last joint frame 6.

Notice: We use $\mathbf{T}_0^N$ (here with N=6) as this is then the same for all robots of this type

The Cartesian space X is defined as the set of all positions and orientations that the outermost link in the robot can be in.

We will here for a robot arm with N joints represent a given position and orientation of the end effector as the *homogeneous transformation* matrix for the "frame in X "

$$\mathbf{T}_0^N = \left[\begin{array}{ccc|c} \text{---} & \mathbf{R}_0^N & \text{---} & \mathbf{p}_0^N \\ 0 & 0 & 0 & 1 \end{array} \right]$$

where $\mathbf{R}_0^N$ is the 3x3 rotation matrix from frame 0 to frame N and $\mathbf{p}_0^N$ contains the positional displacement from frame 0 to frame N measured in the 0-frame.

We now introduce a **parametrization** S_X for X . A parameter in S_X is written as $\alpha_X = (x, y, z, \phi_x, \phi_y, \phi_z)$ where x, y, z simply is the coordinates of the position $\mathbf{p}_0^N$ and ϕ_x, ϕ_y, ϕ_z are so-called Euler angles for $\mathbf{R}_0^N$.

We can get from a parameter α_X to the corresponding frame in X as follows

$$\mathbf{T}_0^N = h(x, y, z, \phi_x, \phi_y, \phi_z) = \left[\begin{array}{ccc|c} C_y C_z & S_x S_y C_z - C_x S_z & C_x S_y C_z + S_x S_z & x \\ C_y S_z & S_x S_y S_z + C_x C_z & C_x S_y S_z - S_x C_z & y \\ -S_y & S_x C_y & C_x C_y & z \\ \text{---} & \text{---} & \text{---} & \text{---} \\ 0 & 0 & 0 & 1 \end{array} \right]$$

where $C_u \equiv \cos(\phi_u)$; $S_u \equiv \sin(\phi_u)$

There are several def's of Euler angles

Cartesian space and configuration space as mathematical sets

We represent a configuration in $\mathcal{C}$ as the set of homogeneous transformations

$$H_T = \{\mathbf{T}_0^1, \mathbf{T}_0^2, \dots, \mathbf{T}_0^N\}$$

where N is the number of joints in the robot and $\mathbf{T}_0^i$ is the transformation from the robot base (0) frame to a frame fixed with respect to the link driven by joint i . Frame i is typically placed on joint axis i with the z -axis pointing along the joint axis. Frame locations are specified by robot vendors (read the manuals).

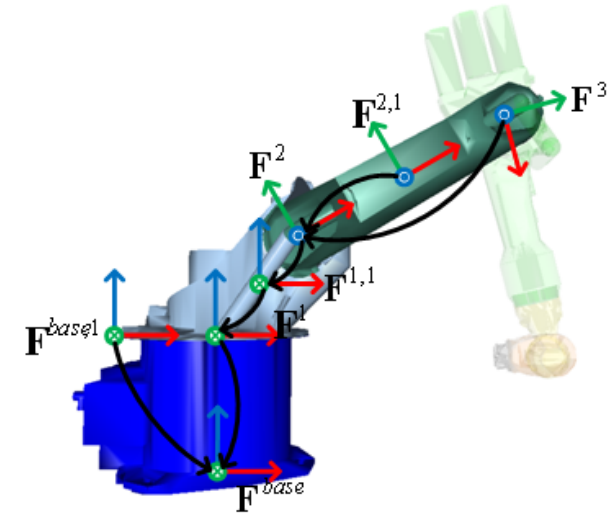
We now introduce a **parametrization** S for $\mathcal{C}$. A parameter in S is written as $\alpha = (\theta_1, \theta_2, \dots, \theta_N)$ where θ_i simply is the value of joint angle i .

We can get from a parameter α to the corresponding frame set H_T in $\mathcal{C}$ using **forward (direct) kinematics**

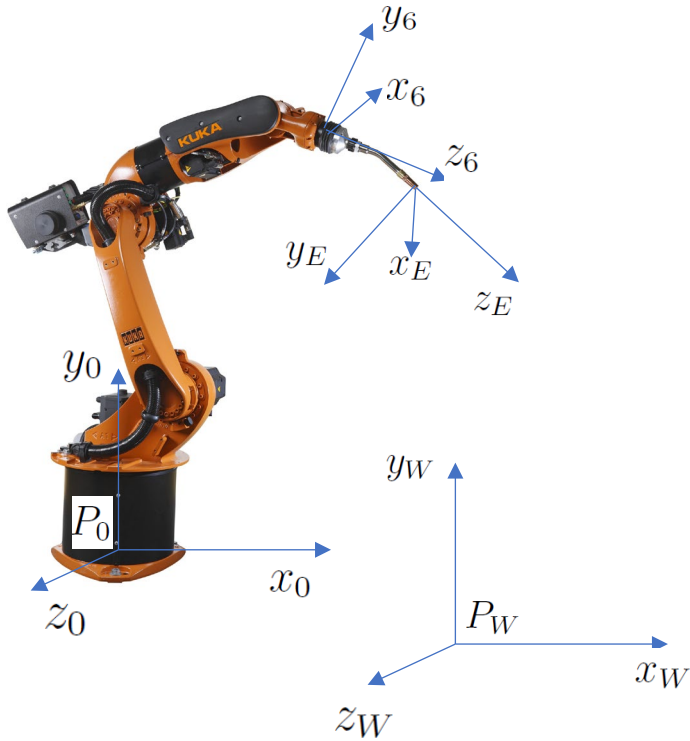
$$H_T = H(\theta_1, \dots, \theta_N) = \{\mathbf{T}_0^1(\theta_1), \mathbf{T}_0^1(\theta_1)\mathbf{T}_1^2(\theta_2), \dots, \mathbf{T}_0^1(\theta_1)\mathbf{T}_1^2(\theta_2)\dots\mathbf{T}_{N-1}^N(\theta_N)\}$$

The homogeneous transformation matrices $\mathbf{T}_{i-1}^i(\theta_i)$ relate frame $\mathbf{F}_{i-1}$ fixed in link $i-1$ to frame $\mathbf{F}_i$ fixed in link i and is analytically computable based on the robot kinematics. There are different formulations - for example using so-called Denavit-Hartenberg parameters. Here, it is enough to know that they are known **robot specific** analytic functions.

The *configuration space* $\mathcal{C}$ is defined as the set of all configurations that the robot can be in. Each point in $\mathcal{C}$ **represents a set of positions and orientations of each of the rigid components that are feasible with respect to the joint constraints.**



Cartesian space and configuration space. Inverse Kinematics



Let us now look at the other way around. Given coordinates $(x, y, z, \phi_x, \phi_y, \phi_z)$ in S_x and the corresponding frame $h(x, y, z, \phi_x, \phi_y, \phi_z)$ in X . Find the set of $(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6)$ that hits this frame (inverse kinematics). We may write this problem as

$$\mathbf{T}_0^1(\theta_1)\mathbf{T}_1^2(\theta_2) \dots \mathbf{T}_5^6(\theta_6) = h(x, y, z, \phi_x, \phi_y, \phi_z)$$

Often $\mathbf{T}_W^E$ is the input. What then?

There may be zero, or several solutions. Among the different robot brands, there is no systematic way of finding these solutions. In many cases, there are zero, four or eight solutions to the inverse kinematics. We will just assume that we have these and write them as $k \geq 0$ inverse kinematics solutions

$$\begin{aligned} &\alpha^{(1)}(x, y, z, \theta_x, \theta_y, \theta_z) \\ &\alpha^{(2)}(x, y, z, \theta_x, \theta_y, \theta_z) \\ &\dots \\ &\alpha^{(k)}(x, y, z, \theta_x, \theta_y, \theta_z) \end{aligned}$$

where each solution $\alpha^{(i)}$ contains solution values of the 6 joint variables $\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6$.



Cartesian space and configuration space. Inverse Kinematics

Let us now look at the other way around. Given coordinates $(x, y, z, \phi_x, \phi_y, \phi_z)$ in S_x and the corresponding frame $h(x, y, z, \phi_x, \phi_y, \phi_z)$ in X . Find the set of $(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6)$ that hits this frame (inverse kinematics). We may write this problem as

$$\mathbf{T}_0^1(\theta_1)\mathbf{T}_1^2(\theta_2) \dots \mathbf{T}_5^6(\theta_6) = h(x, y, z, \phi_x, \phi_y, \phi_z) \equiv [\mathbf{T}_W^0]^{-1}\mathbf{T}_W^E[\mathbf{T}_6^E]^{-1}$$

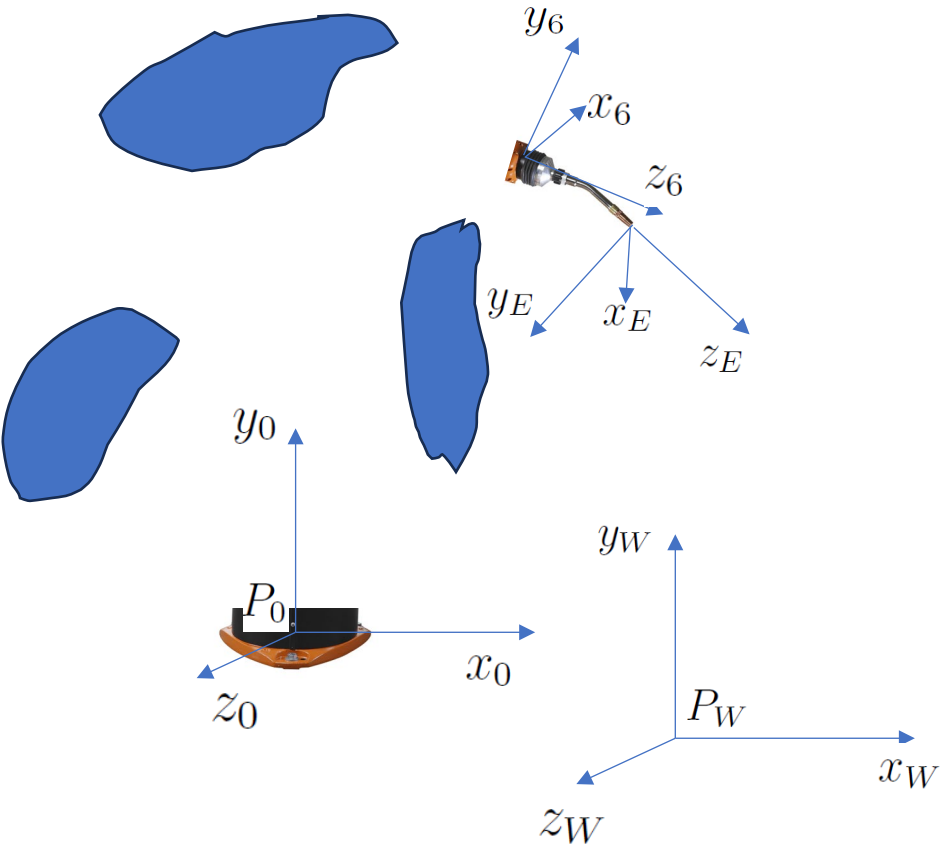
There may be zero, or several solutions. Among the different robot brands, there is no systematic way of finding these solutions. In many cases, there are zero, four or eight solutions to the inverse kinematics. We will just assume that we have these and write them as $k \geq 0$ inverse kinematics solutions

$$\begin{aligned} &\alpha^{(1)}(x, y, z, \theta_x, \theta_y, \theta_z) \\ &\alpha^{(2)}(x, y, z, \theta_x, \theta_y, \theta_z) \\ &\dots \\ &\alpha^{(k)}(x, y, z, \theta_x, \theta_y, \theta_z) \end{aligned}$$

where each solution $\alpha^{(i)}$ contains solution values of the 6 joint variables $\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6$.

Pieper's theorem for 6-axis robots:
When three consecutive axes meet at a point, or when three consecutive axes are parallel, there is always a method to analytically derive all inverse kinematics solutions.

- Spherical wrist (e.g. Kuka, ABB above)
- Three consecutive parallel axes (UR) (meeting point at "infinity" also works)



Task Specification

Task Trajectory Generation

Collision Checking

Planning Collision Free Motions

Tasks are specified as end effector configurations in the world frame. Hence we know $\mathbf{T}_W^E$.

We assume to know where the robot is placed ($\mathbf{T}_W^0$) and the end effector frame relative to the $N = 6$ 'th frame in the robot $\mathbf{T}_6^E$.

Hence the corresponding frame in X can be computed as $\mathbf{T}_0^6 = [\mathbf{T}_W^0]^{-1} \mathbf{T}_W^E [\mathbf{T}_6^E]^{-1}$

Task trajectories are generated by interpolating in parameter space S_X (lecture later this morning by Aljaz Kramberger).

Finally, we pick one of the inverse kinematics solutions. Hence we have joint coordinates $\theta_1, \dots, \theta_6$ and all robot frames $H(\theta_1, \dots, \theta_6)$.

Then it can be tested if the we have no collisions. Here we can see it, but on the computer we need to deploy a collision checking method.

Planning takes place in the parameter space $S \subseteq \mathbb{R}^N$ for the configuration space $\mathcal{C}$.

Cartesian space X. A “point” in X is the homogeneous transformation

$$\mathbf{T}_0^N = \left[\begin{array}{ccc|c} & \mathbf{R}_0^N & & \mathbf{p}_0^N \\ \hline & & & \\ 0 & 0 & 0 & 1 \end{array} \right]$$

$$h(x, y, z, \phi_x, \phi_y, \phi_z) = \left[\begin{array}{ccc|c} C_y C_z & S_x S_y C_z - C_x S_z & C_x S_y C_z + S_x S_z & x \\ C_y S_z & S_x S_y S_z + C_x C_z & C_x S_y S_z - S_x C_z & y \\ -S_y & S_x C_y & C_x C_y & z \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

Coordinate representation S_x for the Cartesian space X. Point is:

$$\alpha_X = (x, y, z, \phi_x, \phi_y, \phi_z)$$

Trajectory generation (Aljaz)

Collision Checking / Clearance computation (afternoon)

Configuration space C. A “point” in C is the set of homogeneous transformations

$$H_T = \{\mathbf{T}_0^1, \mathbf{T}_0^2, \dots, \mathbf{T}_0^N\}$$

$$H(\theta_1, \dots, \theta_N) = \{\mathbf{T}_0^1(\theta_1), \mathbf{T}_0^1(\theta_1)\mathbf{T}_1^2(\theta_2), \dots, \mathbf{T}_0^1(\theta_1)\mathbf{T}_1^2(\theta_2) \dots \mathbf{T}_{N-1}^N(\theta_N)\}$$

Forward Kinematics

Inverse Kinematics

Coordinate representation S for the Configuration space C. Point is:

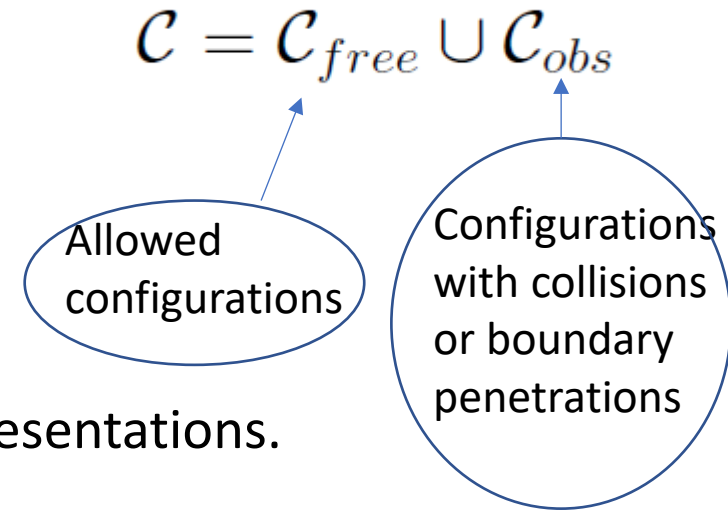
$$\alpha = (\theta_1, \theta_2, \dots, \theta_N) \quad \theta_i^{\min} \leq \theta_i \leq \theta_i^{\max}$$

Interpolation and cycle time estimation (now)

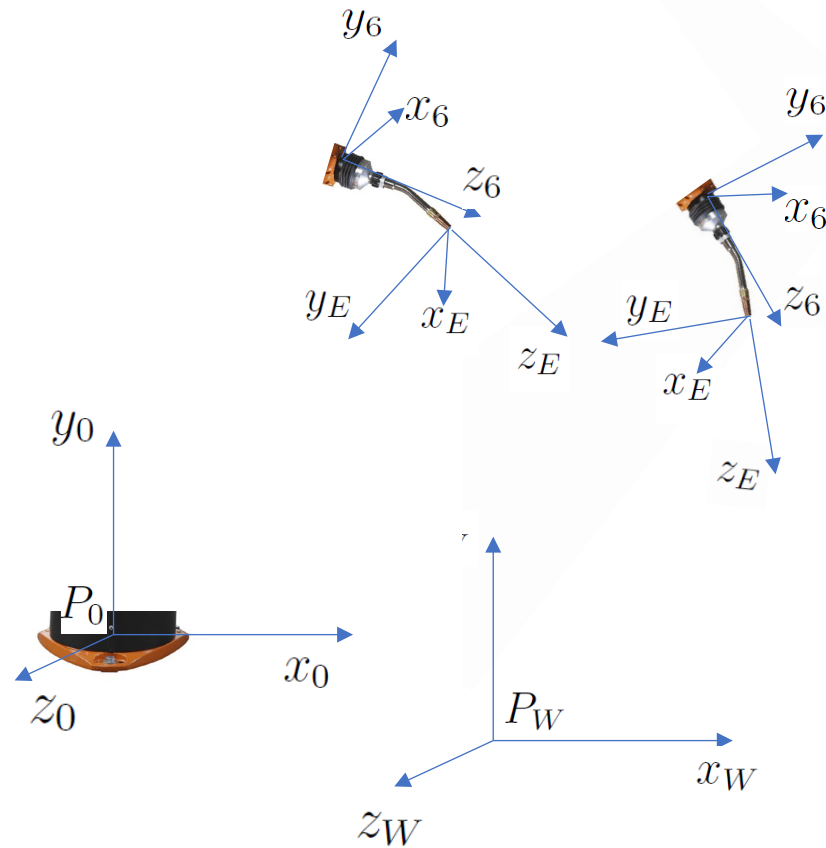
Planning collision free robot paths (afternoon)

Robot Tasks and Planning collision free paths

- Robot tasks are specified in Cartesian space
- Planning is performed in configuration space
- Forward and inverse kinematics are used to move between the two representations.
- The most simple planning problem is to plan a collision free path between **two given collision free points in configuration space** (selected inverse kinematics solution). This is called the "point to point (motion) planning problem (precise definition in the afternoon)".
- Outcome is typically a discrete sequence of points in configuration space representing the path where the linear segments between adjacent points have been checked to be collision free (**afternoon**)



Exercise during the break:



What is the shortest possible robot path between two end effector configurations? (Collision problems can be neglected.)

Estimating execution time costs

Robot controller manuals typically provide maximal joint speeds

$$|\dot{\theta}_k| \leq v_k^{max}; \quad k = 1, \dots, N$$

We now estimate a lower bound on the time it takes to execute a linear joint path between α^A and α^B as

$$t(\alpha^A, \alpha^B) = \max_k \left\{ \frac{|[\alpha^A]_k - [\alpha^B]_k|}{v_k^{max}} \right\}$$

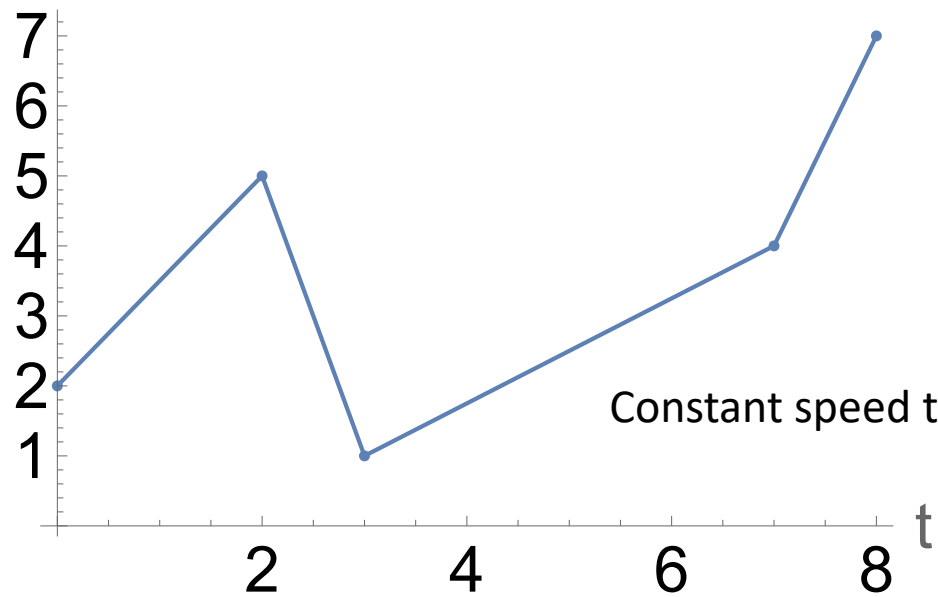
Assume now that we have a solution passing through

$$\alpha^{init} \equiv \alpha^{(1)}, \alpha^{(2)}, \alpha^{(3)}, \dots, \alpha^{(M)} \equiv \alpha^{goal}$$

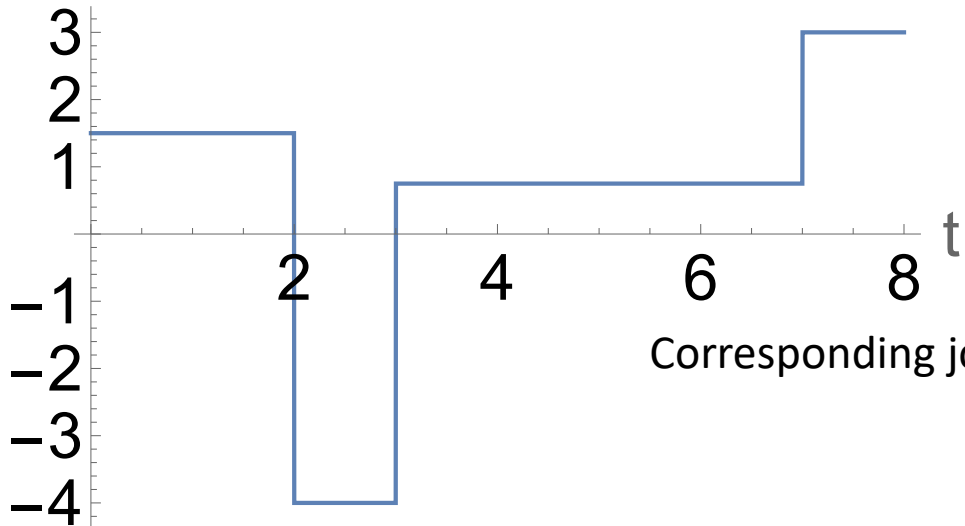
Then a lower bound on the execution time is given as

$$T_{low} = \sum_{n=1}^{M-1} t(\alpha^{(n)}, \alpha^{(n+1)})$$

Why is this a lower bound and not an estimate of the execution time?

$[\alpha]_k(t)$ 

Constant speed trajectory of a joint as function of time

 $[\alpha]_k'(t)$ 

Corresponding joint velocity.

We need infinite accelerations to carry out this movement. Actually a full stop is required at the corner points to avoid infinite accelerations.

Parabolic blends

Consider two vector valued constant speed linear segments that intersect at $\mathbf{X}$ at $t = T$ where t is time.

$$\begin{aligned}\mathbf{x}_1(t) &= \mathbf{X} + \mathbf{v}_1(t - T) & t \leq T \\ \mathbf{x}_2(t) &= \mathbf{X} + \mathbf{v}_2(t - T) & t \geq T\end{aligned}$$

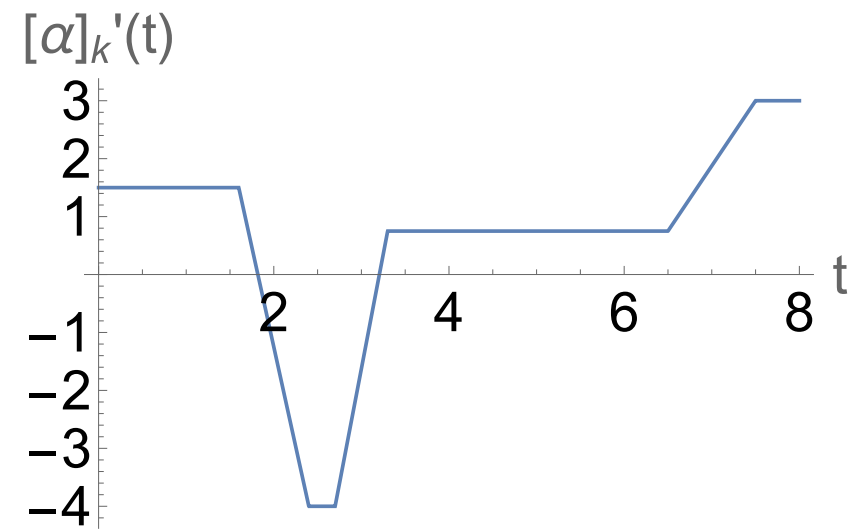
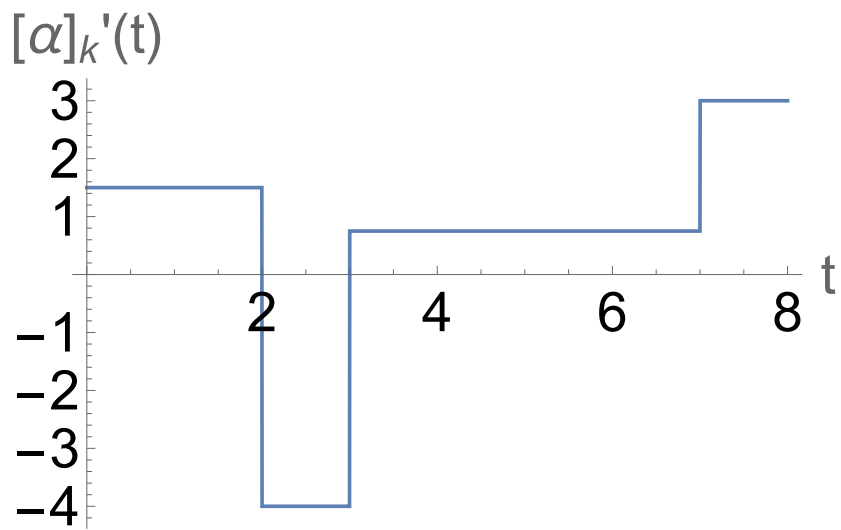
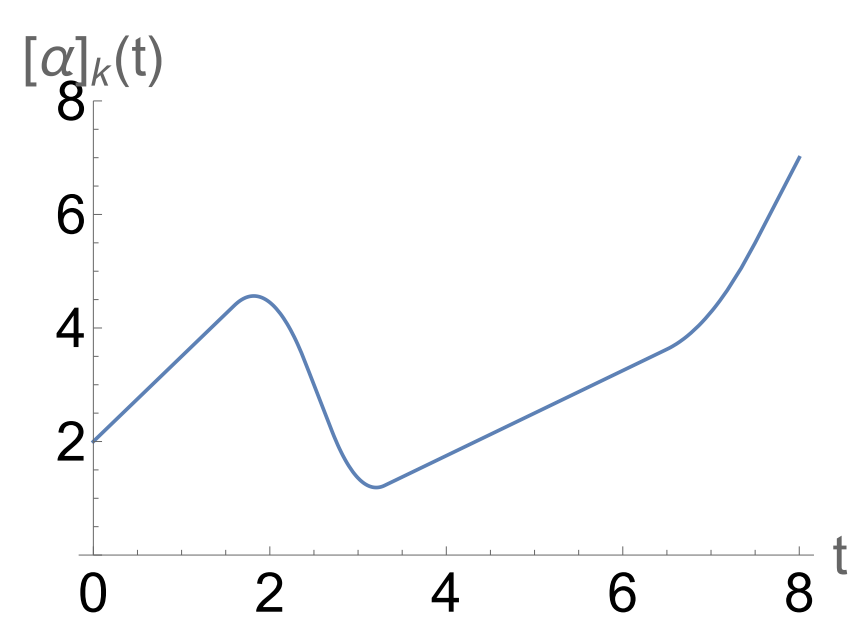
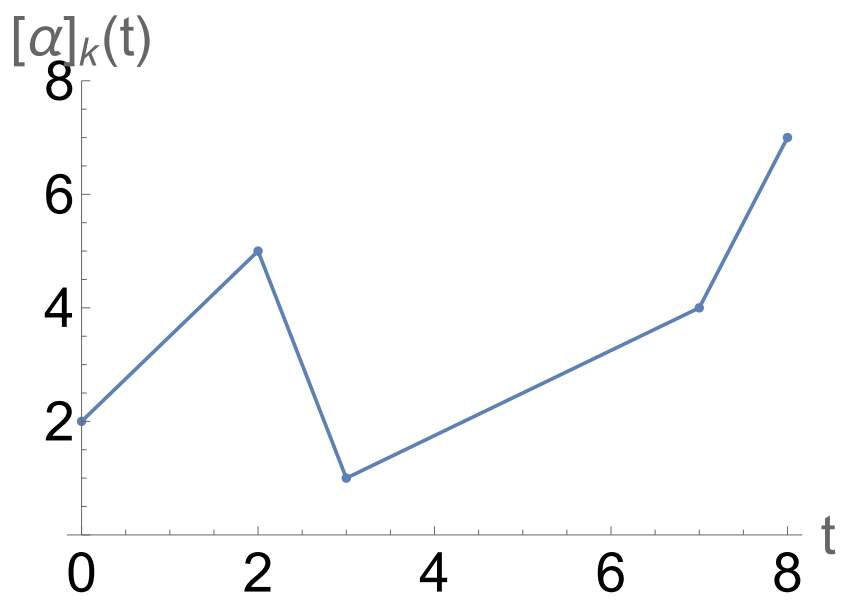
For an arbitrarily chosen blend interval $T - \tau \leq t \leq T + \tau$, we may then define a parabola that blends from $\mathbf{x}_1(t)$ to $\mathbf{x}_2(t)$. The parabola is defined as

$$P(t - T, \mathbf{X}, \mathbf{v}_1, \mathbf{v}_2, \tau) = \frac{\mathbf{v}_2 - \mathbf{v}_1}{4\tau}(t - T + \tau)^2 + \mathbf{v}_1(t - T) + \mathbf{X}$$

Notice that the acceleration is constant and given by $\frac{\mathbf{v}_2 - \mathbf{v}_1}{2\tau}$. The value of τ must be chosen by the user.

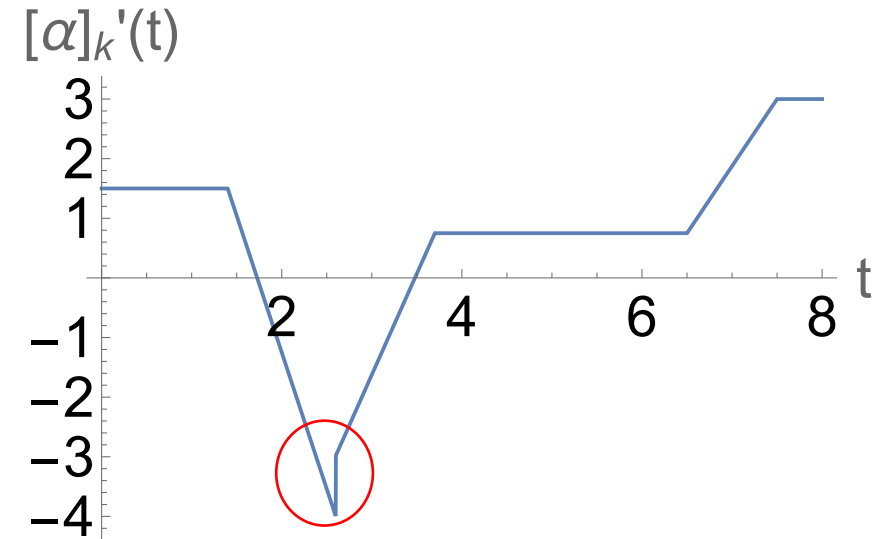
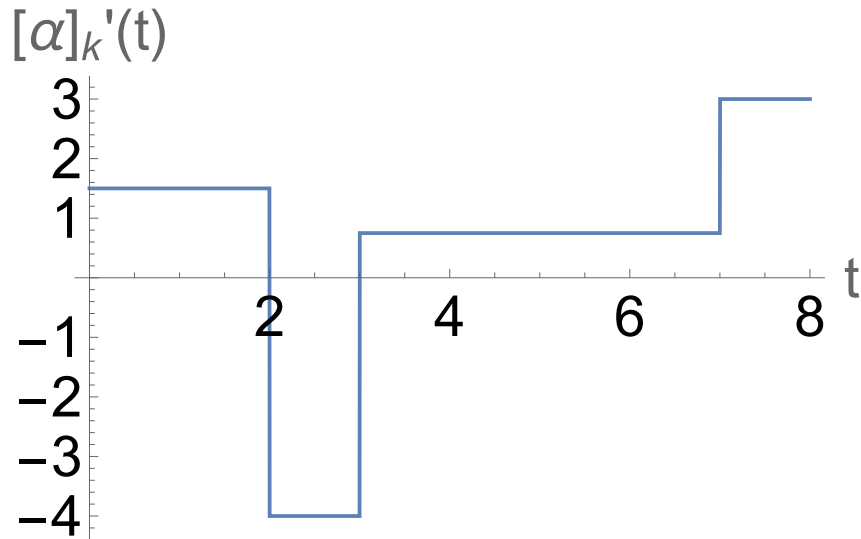
For the choice of τ , two opposing criteria are in play: i) We wish to limit the acceleration during the blend, which puts a lower limit on τ , and ii) We wish to minimize the distance of the parabola to the two linear segments.

The distance between the original and the blended trajectory is largest at $t = T$, where we by insertion obtain a displacement $\mathbf{d} = \frac{1}{4}(\mathbf{v}_2 - \mathbf{v}_1)\tau$.



Notice that the execution time (8 seconds) remains the same with blending.
Is this always the case or were we just lucky?

Parabolic blends – constraints on blend time τ .



Consecutive blend intervals are not allowed to overlap. Upper limit on τ .

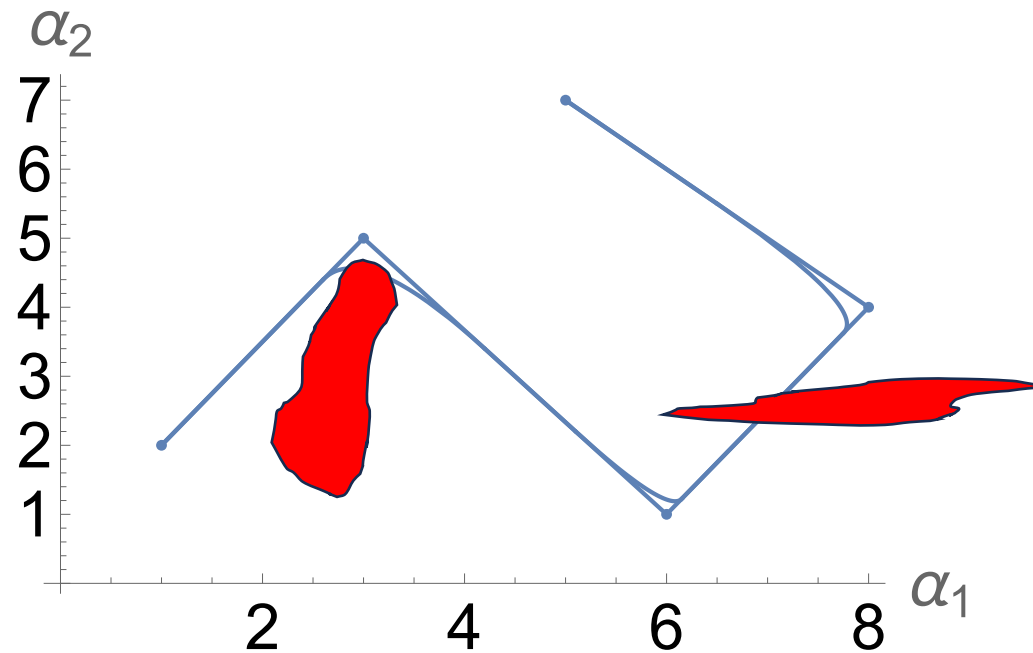
The blend acceleration $\mathbf{a} \equiv \frac{\mathbf{v}_2 - \mathbf{v}_1}{2\tau}$ must be achievable by the robot. This puts a lower limit on τ .

If the upper limit is smaller than the lower limit, we have a problem. What to do then?

Reduce speed (time scaling)

Other constraints to consider?

Blended curves in configuration space



The original curve was collision free, but the blended curve is in collision.

We have not even yet considered if the original curve was collision free.

Topics for the afternoon session.

Why is it best to interpolate in configuration space for point to point planning

- Shortest path between any two configurations is a linear path in configuration space.
- Lowest level control of a robot are by controlling joint movements – not Cartesian movements as the motors are moving the joints
- A simple curve in Cartesian space (e.g. straight line) leads to a complex curve in configuration space, and hence many waypoints, which again leads to limitations in speed.
- Cycle time estimation is easier for simple curves in configuration space
- Collision avoidance is a problem formulated in configuration space
- Interpolating in Cartesian space can lead to “*singularity*” problems (next slide).
- Hence, interpolation in Cartesian space should only be used for tasks where the end effector path is pre-specified. For example, welding, milling, grinding, polishing, etc.

Singularities

$$\mathbf{T}_0^1(\theta_1)\mathbf{T}_1^2(\theta_2)\dots\mathbf{T}_5^6(\theta_6) = h(x, y, z, \phi_x, \phi_y, \phi_z)$$

Linearization of this equation:

$$\mathbf{J}(\alpha)d\alpha = (dx, dy, dz, d\omega_x, d\omega_y, d\omega_z)^T \quad \alpha = (\theta_1, \theta_2, \dots, \theta_N) \quad (N=6)$$

Jacobian (6xN matrix)

Singularities occur when the
Jacobian loses rank (is singular)

Interpolations in Cartesian space can lead to singularities

Interpolations in configuration space remove the singularity problems

Concepts that you should surely remember

- Cartesian space and configuration space
- Parametrization for Cartesian space and for configuration space
- Homogeneous transformation matrices and their content
- Forward kinematics and Inverse kinematics
- Velocity constraints and their impact on the lower bound on cycle time for the path between two joint configurations
- Acceleration constraints and their importance for blend window sizes
- For "air movements" prefer linear joint movements over linear or circular Cartesian movements

Notice: Objectives for trajectory optimization depends on company wishes: Cycle time, energy consumption, wear and tear,... Or a combination of several objectives

Robotic Palletizing Video

Try to critically observe as much as you can concerning the robot trajectories

<https://www.youtube.com/watch?v=O6CjDfmBXHU>

Exercise during the break: Outline a concept for how to program a single pick and place for a palletizing execution starting at the pick and ending at the placement.

The concept should include a small set of Cartesian frames including the pick and place frame, and how you would interpolate between these.

If relevant you may say that you pick inverse kinematics solutions to one or more of the Cartesian frames with a little statement of which solution to pick.

Aug. 12th : Planning of collision free and feasible robot motions

08:30 – 10.15 Lecture 1: Basic planning and trajectory concepts (HP)

- Cartesian space and configuration space
- Parametrizations of these spaces
- Mapping between spaces
- Linear interpolation with parabolic blends
- Estimating cycle time costs
- Why plan and interpolate in configuration space

10:30 – 12.00 Lecture 2: Trajectory planning for Robot Manipulators and Grasping (AK)

- Interpolation for trajectories represented in Cartesian space
- Cubic polynomial interpolation
- Robotic grasping
- Defining trajectories by Learning

12:00 – Lunch break

13:00 – 14.30 Lecture 3: Finding collision free paths (HP)

- Point to point planning
- Distance metric
- Resolution property
- Grid based and random points based sampling
- Sample based planning methods in a nutshell
- Collision testing and distance computations
- Finding shortest paths in graphs
- Example of a planning method

14:30 – Coffee break

15:00 – 16.30 Lecture 4: Behavior based robotics for easy programming (KS)

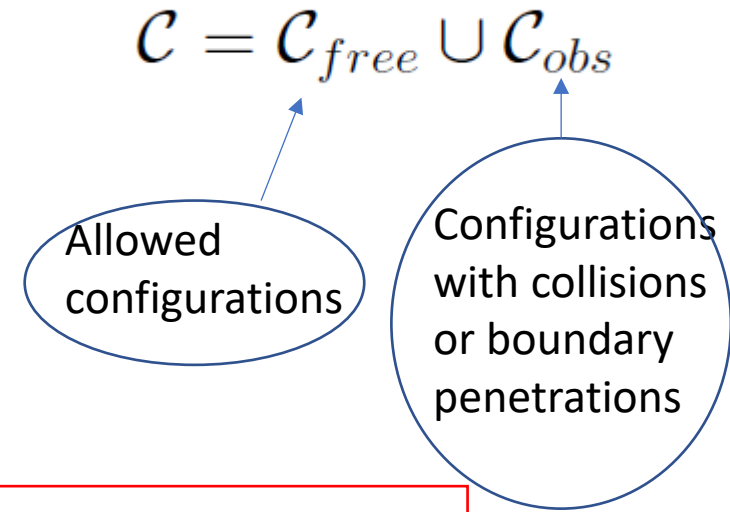
16:30 – Project work

Point-to-point motion planning problem

We now introduce a **parametrization** S for $\mathcal{C}$. A parameter in S is written as $\alpha = (\theta_1, \theta_2, \dots, \theta_N)$ where θ_i simply is the value of joint angle i .

We can get from a parameter α to the corresponding frame set H_T in $\mathcal{C}$ using **forward (direct) kinematics**

$$H_T = H(\theta_1, \dots, \theta_N) = \{\mathbf{T}_0^1(\theta_1), \mathbf{T}_0^1(\theta_1)\mathbf{T}_1^2(\theta_2), \dots, \mathbf{T}_0^1(\theta_1)\mathbf{T}_1^2(\theta_2)\dots\mathbf{T}_{N-1}^N(\theta_N)\}$$



Consider a configuration space $\mathcal{C}$ with parametrization α . We now define S_{free} and S_{obs} as the sets of coordinates α where $H(\alpha) \in \mathcal{C}_{free}$ and $H(\alpha) \in \mathcal{C}_{obs}$ respectively. Consider two configurations $\alpha^{init}, \alpha^{goal} \in S_{free}$. Find a continuous path in S_{free} between the two configurations.

Distance metric

Although it is not strictly necessary, it is in general useful to define a distance metric $d(\alpha^A, \alpha^B)$ on S . It can be any metric, e.g. the simple unweighted 2-norm, but for efficiency it is desirable to define a metric based on movements in Cartesian space.

This can be made arbitrarily advanced, but a simple metric that will do the job is to choose e.g.

$$d(\alpha^A, \alpha^B) \equiv \sqrt{\sum_{i=1}^6 [w_i(\alpha_i^A - \alpha_i^B)]^2} \quad (\text{weighted 2-norm})$$

$$d(\alpha^A, \alpha^B) \equiv \sum_{i=1}^6 w_i |\alpha_i^A - \alpha_i^B| \quad (\text{weighted 1-norm})$$

where w_i 's are strictly positive weights defined by estimates of the robot travel in Cartesian space when moving parameter α_i a unit step while fixing all other parameters fixed. Notice that $d(\alpha^A, \alpha^B)$ has units in metres.

Literature suggests other weights as well, but none are any better...

Resolution property

Assume that S_{free} is a finite open subset of $\mathbb{R}^N$ (it can be made open by taking the closure on all obstacles and the boundary). Consider now arbitrary $\alpha^{init}, \alpha^{goal} \in \mathcal{C}_{free}$ where there is a solution to the point to point motion planning problem.

Then there exists a $\delta > 0$ (the needed resolution) satisfying the following

Consider any "coverage" of S_{free} with a finite set, P , of points satisfying that for any point $\alpha \in S_{free}$, there exists a $p \in P$ so that $d(\alpha, p) < \delta$. Then we can always find a sequence of collision free points $\alpha^{init}, p_1, \dots, p_m, \alpha^{goal}$ where $p_i \in P$; $i = 1, \dots, m$, and all straight lines between adjacent points in the sequence are collision free.

Basis for "Sample based planning": **Insert points in S and try to connect them while checking for collisions.**
Different strategies for all that with different planners.

You may assume that **you have a function that takes a point in S as input and returns if it is in collision, and if it not, it also provides a closest distance to an obstacle.** NOTICE: Hence, you can choose to use a minimal clearance

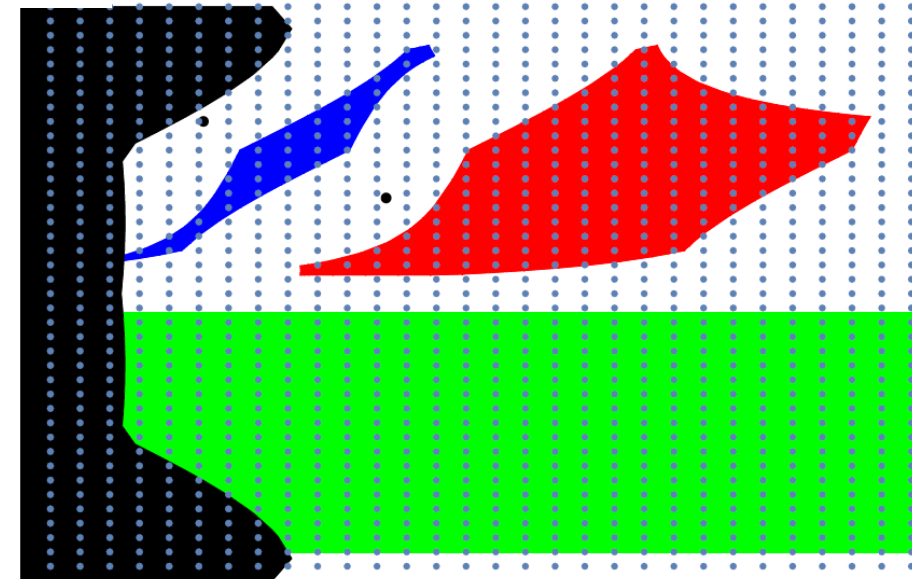
Grid based planning

Choose a gridsize D and place a grid on S (nearest neighbours α^A, α^B satisfies $d(\alpha^A, \alpha^B) = D$). Search for collision free paths using the grid points and straight lines between nearest neighbours.

Advantage: If $\frac{\sqrt{N}}{2}D \leq \delta$, we know that a solution can be found if it exists.

Disadvantage: The value of δ is unknown and for small D 's, the grid is huge. Consider for example $S = (-\pi; \pi)^6$ and the weights $w = (1, 1, 0.5, 0.3, 0.3, 0.1)$ and we choose $D = 1\text{cm}$. Then we would get approximately 8.7×10^{12} points. If we choose $D = 1\text{mm}$, it would then be 8.7×10^{18} points.

Notice: Rectangular grids are not optimal for this. (Hyper)sphere packing grids can be used. In particular for many dimensions. But search and book keeping in sphere packing grids are more complicated. Furthermore, the number of points will still be extremely high for good resolutions.



Planning based on random points

Insert m random points in S . Search for collision free paths using the random points and straight lines between them.

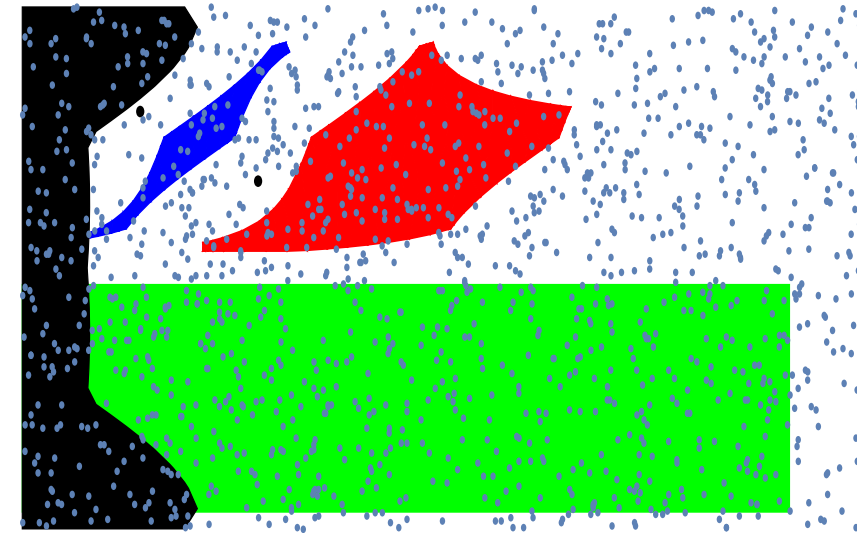
Advantage: As the number of random points goes to infinity, the probability that a given solution can be found with the set of random points goes to one.

Disadvantage: If we for a given number of random points (which could be huge) fail to find a solution, we still don't know if one exists.

NOTICE: Advantages and disadvantages for grid based and random based sampling are conceptually quite similar. What in practise will make the difference is the heuristics used to sample the points and which points to connect.

What is critical is the number of collision checks we do before we have found a solution or DECIDE that we give up on finding one.

No prior knowledge about configuration space. We can only check if a sample point is in collision/compute distance.



Sample based Point to point planning in a nutshell

- 1) Insert samples (include query points)
- 2) Maybe perform collision checks/distance computations to prune samples
- 3) Connect "neighbour" samples to obtain a graph (tree)
- 4) Maybe perform collision checks/distance computations to prune edges
- 5) Find (shortest) graph path between the query points.
- 6) Check graph path for collision
 - Path is collision free 😊
 - If a collision is found, remove some relevant samples and/or edges, and go to 5)
- 7) If no more paths, enhance with new samples and go to 2)
- 8) Until "give up"

NOT EVERY PLANNING ALGORITHM RUNS EXACTLY LIKE THIS, BUT IT COVERS MOST

Sample based Point to point planning in theory and practise

- In theory: If a solution exists, a planning algorithm should find it with probability one in the limit as computer time goes to infinity (completeness).
- In practise:
 - Find an efficient (short) collision free path
 - Use as little computer time as possible to find it
- Computer time is roughly proportional to the number of collision checks/distance computations.

Two types of the point to point planning problem

- "Single" query: A single (or very few) collision free start and end configurations in a given scene. Let us assume just a single $\alpha^{init}, \alpha^{goal}$
- Multiple query: Many collision free start and end configurations in the same scene.
- For single query, heuristics biased towards the given start and end configurations are typically most efficient
- For multiple query, exploration of S for collision free roadmaps can be worthwhile.
- There are a vast amount of different planners for different situations.
- The Open Motion Planning Library (OMPL) <https://ompl.kavrakilab.org/> is recommended.

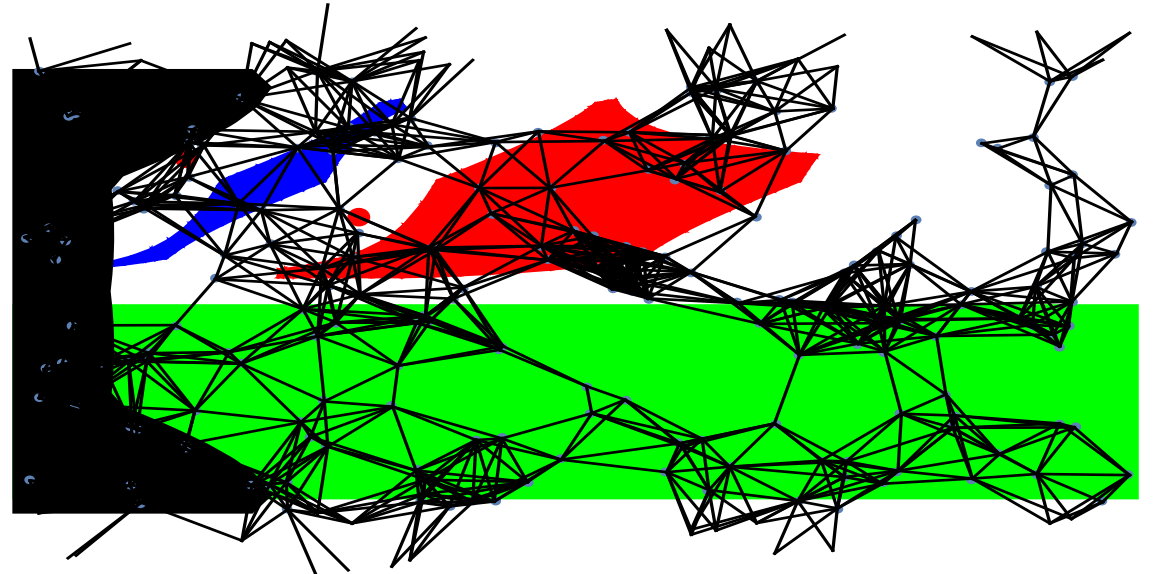
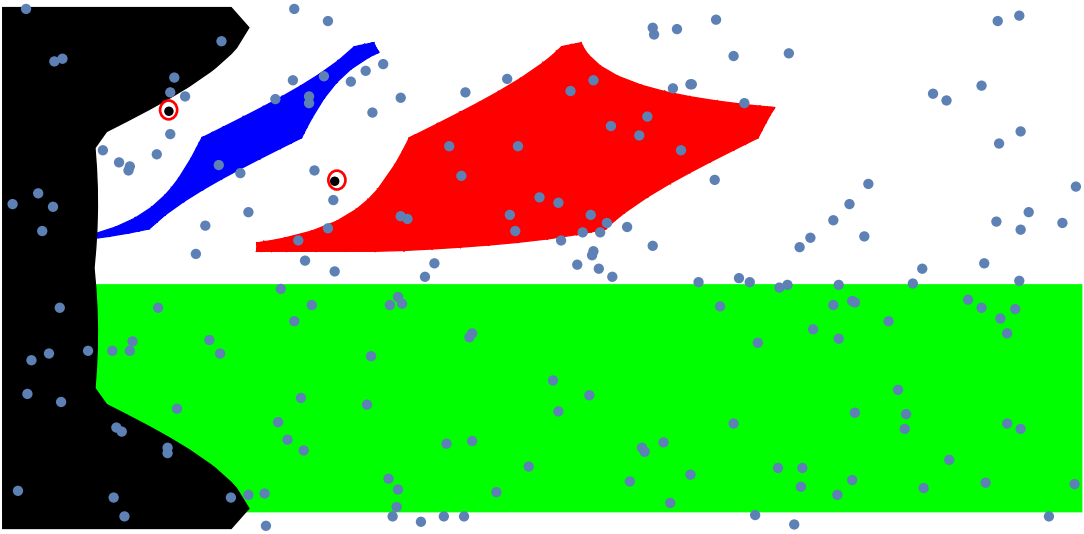
Lazy Probabilistic Roadmaps

R. Bohlin and L. Kavraki, Path Planning Using Lazy PRM,
Proc. 2000 IEEE Int'l Conf. on Robotics and Automation (ICRA 2000).

Select N_{init} random points in S . (Including $\alpha^{init}, \alpha^{goal}$)

User chooses a desired average number of neighbors for a given node M_{neighb} .

Connect two points α^A, α^B with an edge if $d(\alpha^A, \alpha^B) < R_{neighb}$ where R_{neighb} is chosen so that there is on average around M_{neighb} neighbor points to a given point.



Lazy Probabilistic Roadmaps

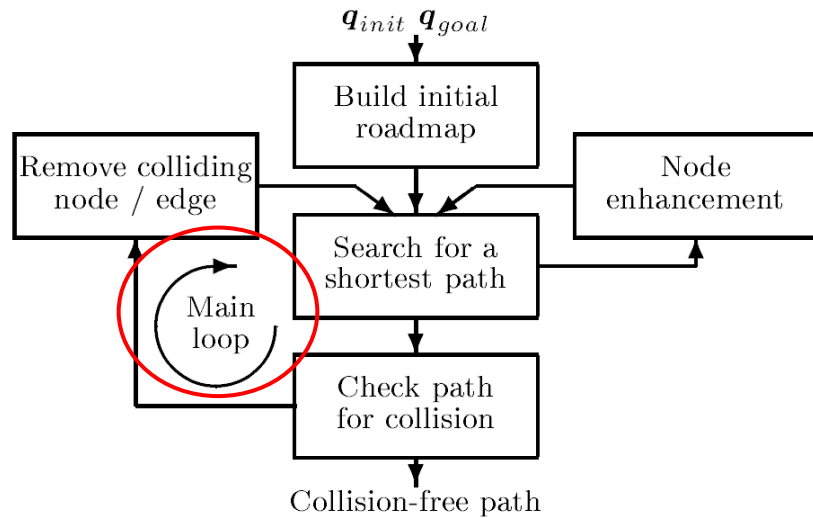
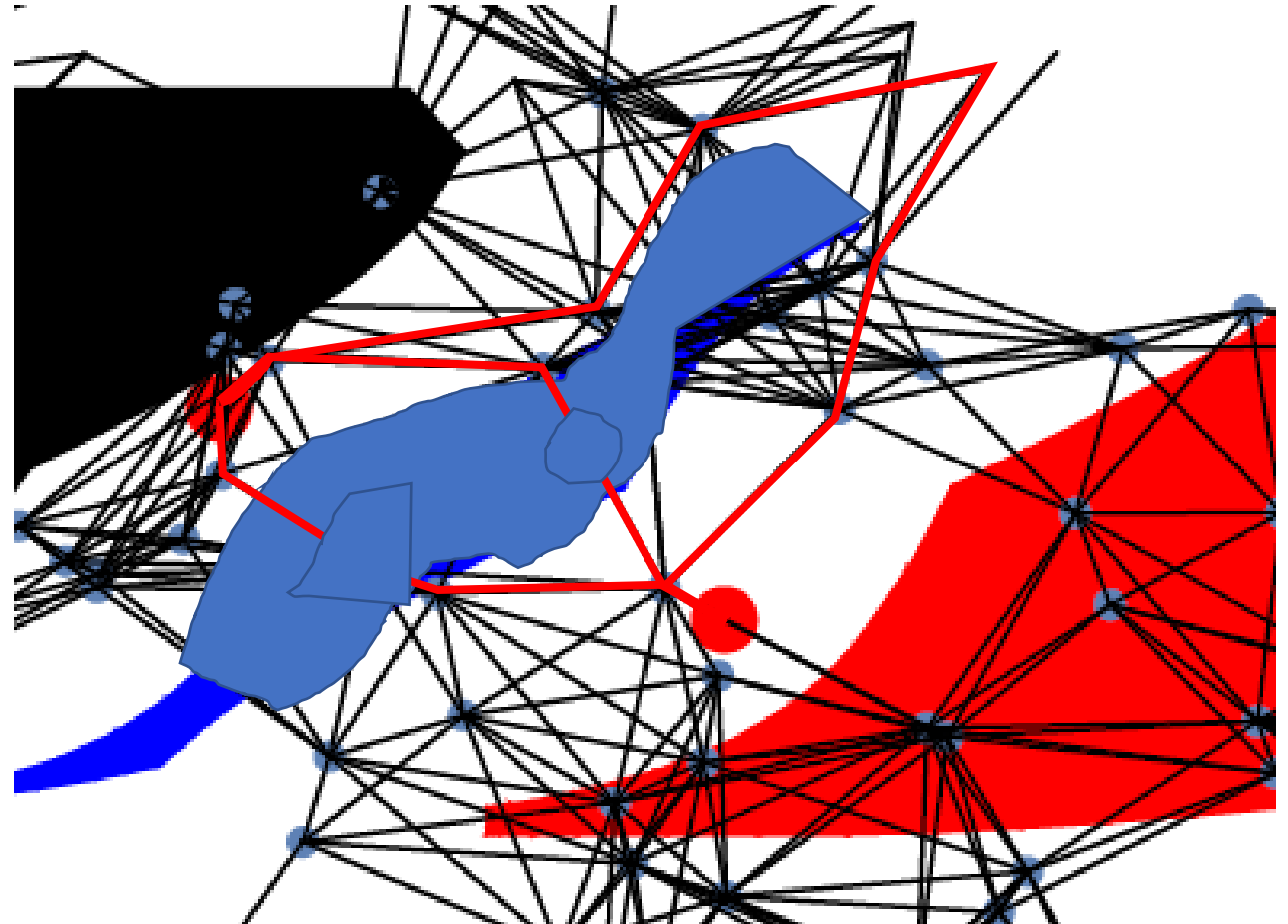
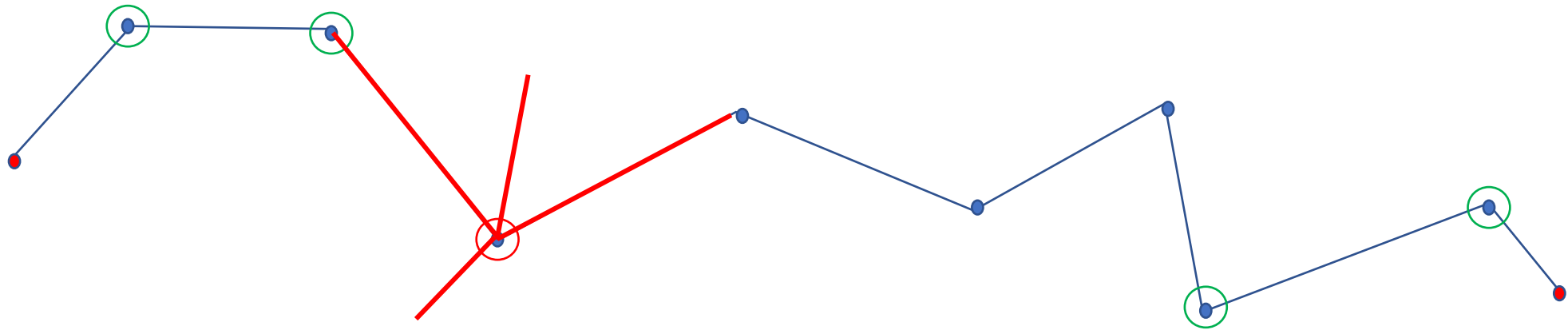


Figure above taken from the paper by Bohlin&Kavraki



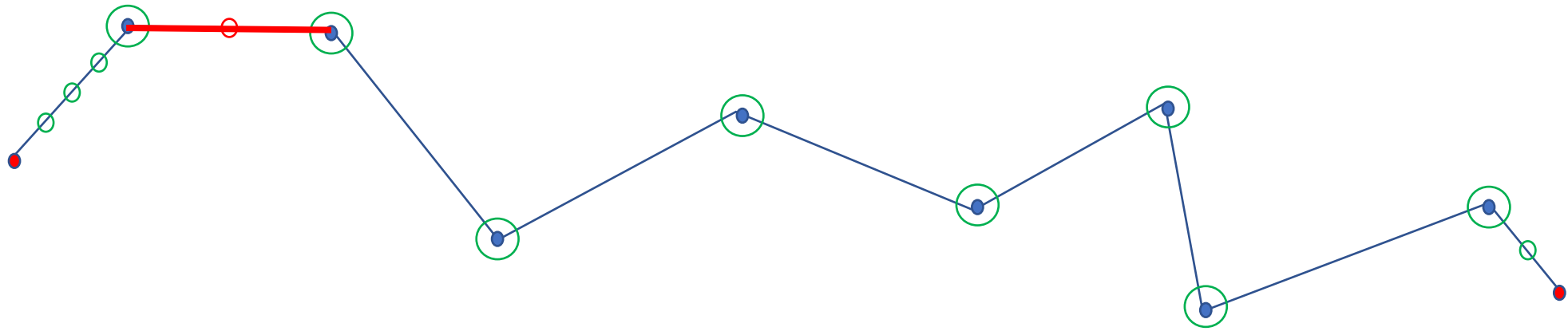
Find shortest path using A*
Vertex on shortest path in collision:
Remove vertex and all edges from the vertex
Find shortest path using A*
Edge in collision:
Remove edge
...
Path found !!!

Checking vertices and edges for collision



Check vertices first. Start with vertices nearest to q_{init} and q_{Goal} . Mark the checked collision free vertices so that they are not rechecked (here green). A colliding vertex and all its edges is marked with collision.

Checking nodes and edges for collision



If all vertices are OK. Then check edges. Start with edges nearest to qInit and qGoal and a coarse resolution.

Subsequently refine to the desired resolution ϵ . Mark the checked collision free points so that they are not rechecked (here green).

Multiple query: Lazy Probabilistic Roadmaps

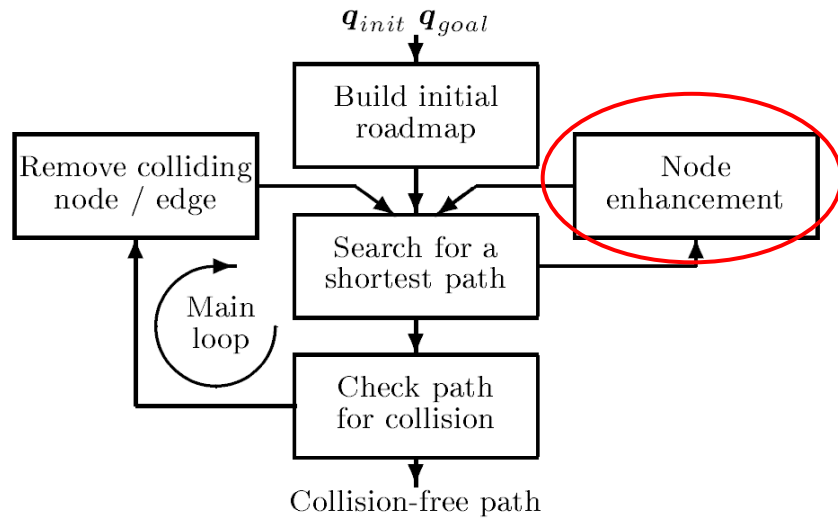
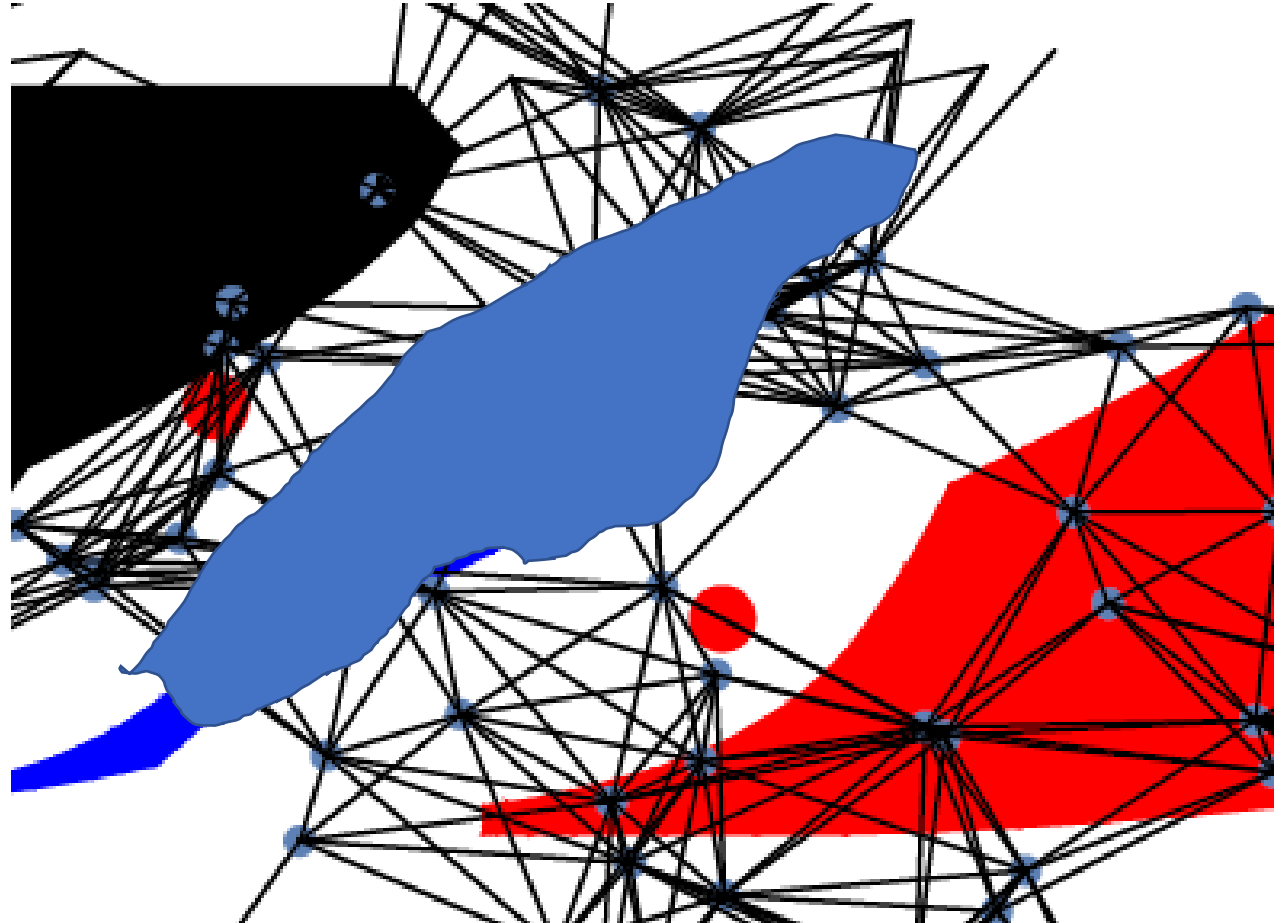


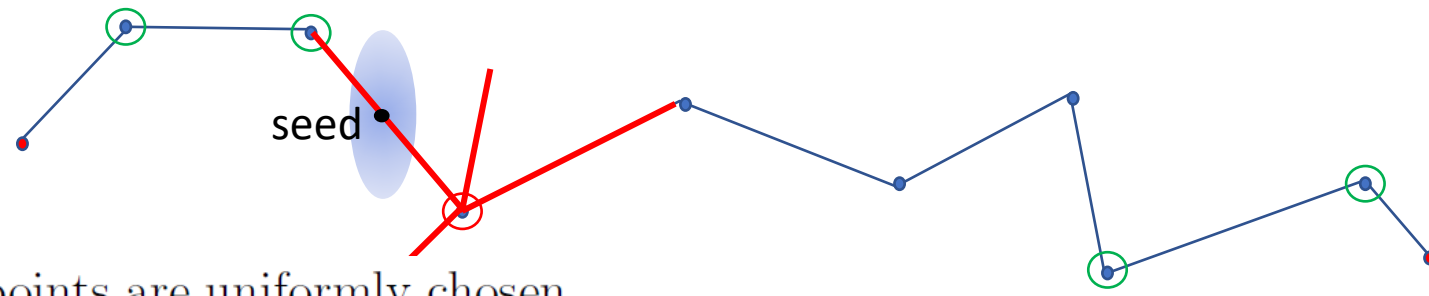
Figure above taken from the paper by Bohlin&Kavraki

Roadmap disconnected between q_{init} and q_{goal} !!!

Node Enhancement !!



Enhancement step



Select N_{enh} new random points in S . Half of the points are uniformly chosen (as the N_{init} initial points). The other half of the points are chosen based on *seeds*.

The set of seeds are selected as the set of midpoints of all removed edges where at least one of the endpoints (nodes) was collision free.

Only edges where both end points were selected from the uniformly distributed sets are considered for seeds (avoids oversampling of regions during subsequent enhancements).

Select the $N_{enh}/2$ seeds randomly from the list and for each seed η , select a new node q taken from the multivariate normal distribution in N_{dof} dimensions centered at η with covariance matrix

$$\Sigma = \frac{R_{neighb}^2}{\chi_{N_{dof}}^2(s)} W^{-1}$$

where W is the (diagonal) matrix containing the distance weights and by construction the probability of the new node being within R_{neighb} from the seed is $1 - s$. A value of $s = 0.05$ is generally a good choice.

Good news: Code is available.
But parameters must be specified:

N_{init}
 M_{neighb}
 N_{enh}
 s
 ϵ

Exercise to consider during the break

Assume we have a scene where we will perform multiple queries, i.e. many $(\alpha^{init}, \alpha^{goal})$'s in the same scene

Which conceptual modifications would you suggest to Lazy PRM?

Sample based Point to point planning in a nutshell

- 1) Insert samples (include query points)
- 2) Maybe perform collision checks/distance computations to prune samples
- 3) Connect "neighbour" samples to obtain a graph (tree)
- 4) Maybe perform collision checks/distance computations to prune edges
- 5) Find (shortest) graph path between the query points.
- 6) Check graph path for collision
 - Path is collision free 😊
 - If a collision is found, remove some relevant samples and/or edges, and go to 5)
- 7) If no more paths, enhance with new samples and go to 2)
- 8) Until "give up"

NOT EVERY PLANNING ALGORITHM RUNS EXACTLY LIKE THIS, BUT IT COVERS MOST

Sample based Point to point planning in a nutshell

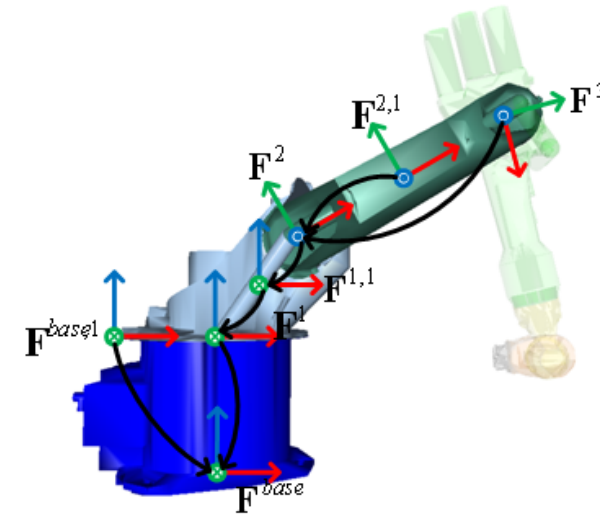
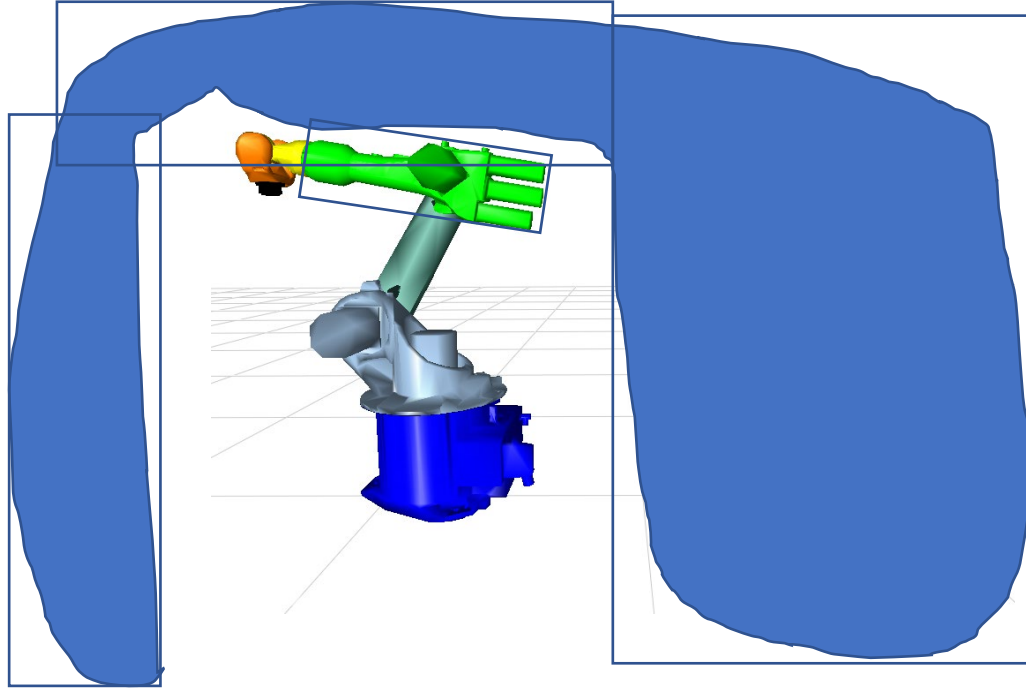
- 1) Insert samples (include query points)
- 2) Maybe perform collision checks/distance computations to prune samples
- 3) Connect "neighbour" samples to obtain a graph (tree)
- 4) Maybe perform collision checks/distance computations to prune edges
- 5) Find (shortest) graph path between the query points.
- 6) Check graph path for collision
 - Path is collision free 😊
 - If a collision is found, remove some relevant samples and/or edges, and go to 5)
- 7) If no more paths, enhance with new samples and go to 2)
- 8) Until "give up"

NOT EVERY PLANNING ALGORITHM RUNS EXACTLY LIKE THIS, BUT IT COVERS MOST

Collision checks and distance computations

- The surface of a link in the robot is typically represented by a "triangle soup", i.e. a set of (many) triangles.
- The coordinates of the triangle corners are given relative to a **chosen geometry frame**, which is fixed with respect to the link.
- For computational efficiency a (typically binary) tree of tight fitting bounding volumes is then created.
- The root of the tree is a tight fitting bounding volumes containing all the triangles of the geometry it represents.
- The children nodes of a given node are created by splitting the triangles into two disjoint sets in an intelligent way and again compute tight fitting bounding volumes for each of the two sets. In this way, a tree of bounding volumes is created where the bounding volumes at the leafs only contain one triangle.
- NOTICE: All bounding volumes will be fixed relative to the **chosen geometry frame**.
- The (typically static) geometry of the environment is created in the same way.

Collision checks and distance computations



To compute the position and orientation of all bounding volumes for a given $\alpha = (\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6)$, we just use H:

$$H(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6) = \{T_0^1(\theta_1) , T_0^1(\theta_1)T_1^2(\theta_2) , \dots , T_0^1(\theta_1)T_1^2(\theta_2) \dots T_5^6(\theta_6)\}$$

Collision checks and distance computations

For collision checks and distance computations, we now exploit the following observation:

Consider bounding volumes from two geometries that we are checking for collisions or computing distances. If the two bounding volumes **do not intersect**, the two sets of triangles that they represent will also not intersect, and the shortest distance between any two triangles (one from each set) will be at least the shortest distance between the two bounding volumes.

This leads to a depth-first search algorithm:

- For collision checking, search down through the two trees until the bounding volumes are collision free. Eventually we are reaching two leaf nodes and if they overlap, their triangles are checked.
- For distance computation, the current shortest distance $d_{\min}$ is kept (initialized to infinity). Search down through the two trees until the distance between the bounding volumes is bigger than $d_{\min}$. Eventually we are reaching two leaf nodes and if their distance is smaller than $d_{\min}$, the shortest distance between their triangles is computed.

Notice: The only reason for depth-first is to try to find collisions as fast as possible if there are any, and for distance computations to get $d_{\min}$ down as fast as possible.

Question: Which two criteria are important when selecting a good type of geometry for the bounding volumes?

Sample based Point to point planning in a nutshell

- 1) Insert samples (include query points)
- 2) Maybe perform collision checks/distance computations to prune samples
- 3) Connect "neighbour" samples to obtain a graph (tree)
- 4) Maybe perform collision checks/distance computations to prune edges
- 5) Find (shortest) graph path between the query points.
- 6) Check graph path for collision
 - Path is collision free 😊
 - If a collision is found, remove some relevant samples and/or edges, and go to 5)
- 7) If no more paths, enhance with new samples and go to 2)
- 8) Until "give up"

NOT EVERY PLANNING ALGORITHM RUNS EXACTLY LIKE THIS, BUT IT COVERS MOST

A* algorithm for finding shortest paths

- Graph (vertices, edges) and two query vertices (v_{init} and v_{goal})
- Edges have positive weights (costs), $d(x,y)$ between vertices x and y . We always have $d(x,x)=0$.
- Fast computable lower bound $h(v)$ on cost from any vertex v to v_{goal}
- Upper bound $g(v)$ on the minimal cost from v_{init} to any other vertex v (initially set to infinity, except that $g(v_{init})=0$).
- The set V consists of points to which a shortest path from v_{init} as already been found.
- Initialize $V=\{v_{init}\}$. $v_{min}=v_{init}$
- One step of the algorithm:
 - Update $g(w)$ for all neighbours w as $g(w)=\text{Min}\{g(w), g(v_{min})+d(v_{min},w)\}$
 - Minimize $f(v)=g(v)+h(v)$ over all vertices not in V . Select v_{min} as the solution.
- The algorithm is terminated when the v_{min} is becoming v_{goal} .

It can easily be shown: *With the reasonable assumption that the triangle inequality $h(x) \leq h(y) + d(x,y)$ holds for the cost function, then any vertex becomes v_{min} at most once.* **Question: Where in the algorithm is this used?**

Question: What is a good fast computable lower bound $h(v)$ for path planning algorithms?

Concepts that you should surely remember

- Point to point planning problem
- Distance metric
- Grid based planning and planning based on random samples
- Single query and Multiple query
- Overall concept of sampling based planning algorithms
- Lazy PRM
- Collision checks/distance computation methods
- A* for finding shortest paths

Aug. 12th : Planning of collision free and feasible robot motions

08:30 – 10.15 Lecture 1: Basic planning and trajectory concepts (HP)

- Cartesian space and configuration space
- Parametrizations of these spaces
- Mapping between spaces
- Linear interpolation with parabolic blends
- Estimating cycle time costs
- Why plan and interpolate in configuration space

10:30 – 12.00 Lecture 2: Trajectory planning for Robot Manipulators and Grasping (AK)

- Interpolation for trajectories represented in Cartesian space
- Cubic polynomial interpolation
- Robotic grasping
- Defining trajectories by Learning

12:00 – Lunch break

13:00 – 14.30 Lecture 3: Finding collision free paths (HP)

- Point to point planning
- Distance metric
- Resolution property
- Grid based and random points based sampling
- Sample based planning methods in a nutshell
- Collision testing and distance computations
- Finding shortest paths in graphs
- Example of a planning method

14:30 – Coffee break

15:00 – 16.30 Lecture 4: Behavior based robotics for easy programming (KS)

16:30 – Project work

Planning Exercise

Optional!!! If you want to work a bit more with the field

Consider a robot planning problem where our dear robot arm is at a current configuration given by the parameter $\alpha^{init} \equiv (\alpha_1^{init}, \dots, \alpha_N^{init})$ and with velocity $\dot{\alpha}^{init} \equiv (\dot{\alpha}_1^{init}, \dots, \dot{\alpha}_N^{init})$. We wish to find an efficient path to one of the two inverse kinematics solutions $\alpha^{goal,A}$ and $\alpha^{goal,B}$ to a given Cartesian goal. The robot should be at rest (have velocity zero) at the goal point.

We first consider an offline planning case where there is a fair amount of time to perform the planning, but where planning should still be as efficient as possible.

- 1) *Outline an algorithm to find a collision free continuous differentiable path with an efficient path execution time.*

Now consider that the planning needs to take place in real time. We first assume that there are no collisions.

- 2) *Outline an algorithm to find a continuous differentiable path with an efficient path execution time. Remember to take the planning time into account.*

The final case is that the planning needs to take place in real time, and there can be collisions. Here, you can assume that $\dot{\alpha}^{init} \equiv \mathbf{0}$

- 3) *Outline an algorithm to find a collision free continuous differentiable path with an efficient path execution time. Again remember to take the planning time into account.*